

ArtConnect Pro

D'une gestion en mémoire à une architecture robuste MySQL

Rapport Technique — Sécurité & Numérique Durable

Base de Données Avancées — Équipe 5

Fahed TADRIST • Hippolyte VALLAT • Arnaud SEKHARA • Roy TAMWO

Année universitaire 2024–2025

Sommaire

1. Introduction et contexte du projet
2. Présentation générale de l'application
3. Modélisation de la base de données
4. Architecture logicielle en couches
5. Sécurité : principes et décisions techniques
6. Numérique durable (Green IT)
7. Fonctionnalités SQL avancées
8. Conclusion et bilan

1. Introduction et contexte du projet

ArtConnect Pro est une application de gestion de communauté artistique locale, développée dans le cadre du cours de Base de Données Avancées à l'eFrei Paris (Paris Panthéon-Assas Université). Le projet, réalisé par l'Équipe 5, a pour objectif central de remplacer un système de persistance en mémoire (InMemory) par une architecture robuste reposant sur une base de données relationnelle MySQL 8.

Au-delà de la simple migration technique, ce projet a été l'occasion d'intégrer dès la conception deux préoccupations transversales majeures de l'informatique moderne : la sécurité des données et l'éco-responsabilité numérique (Green IT). Ce rapport décrit les choix architecturaux, les décisions techniques et les pratiques de développement qui ont permis de répondre à ces enjeux.

2. Présentation générale de l'application

2.1 Objectifs fonctionnels

L'application ArtConnect Pro gère six types d'acteurs et d'entités au sein d'une communauté artistique : les Artistes, les Œuvres (Artworks), les Galeries, les Membres de la communauté, les Ateliers (Workshops) et les Expositions (Exhibitions). Elle offre des opérations CRUD complètes sur chacun de ces domaines, ainsi que des fonctionnalités de recherche, de réservation et de gestion des inscriptions.

2.2 Stack technique

Composant	Technologie
Langage	Java 17
Interface graphique	JavaFX + FXML
Connectivité BDD	JDBC (Java Database Connectivity)
Gestionnaire de build	Apache Maven
Base de données	MySQL 8.x

3. Modélisation de la base de données

3.1 Schéma relationnel

La base de données ArtConnect est structurée en 14 tables couvrant l'ensemble du domaine métier. Elle respecte la Troisième Forme Normale (3FN), ce qui garantit l'absence de redondance et l'intégrité des données. Le passage MCD → MLD → SQL a été réalisé rigoureusement, avec une attention particulière portée aux relations N:N, implémentées via des tables de jonction dédiées.

3.2 Tables principales

- Artiste, Artwork, Gallery, Exhibition, Workshop : entités métier centrales.
- CommunityMember : membres de la communauté, avec un type ENUM FREE/PREMIUM.
- Booking : table de réservation liant Workshop et CommunityMember.
- Discipline, ArtworkTag : tables de référence pour la classification.

3.3 Relations N:N

Deux relations many-to-many sont gérées via des tables de jonction explicites :

- **Appartient** : Appartient : associe un Artist à une ou plusieurs Artworks.
- **Exerce** : Exerce : associe un Artist à une ou plusieurs Disciplines.
- **Reference** : Reference : associe une Artwork à un ou plusieurs ArtworkTags.

Ce choix de modélisation explicite, plutôt qu'une concaténation dans une seule colonne, est une décision de qualité technique qui facilite les requêtes relationnelles et l'évolutivité du schéma.

4. Architecture logicielle en couches

L'application adopte une architecture en couches strictement découplées, inspirée du pattern DAO (Data Access Object). Ce découplage est à la fois une bonne pratique d'ingénierie logicielle et un levier direct pour la maintenabilité et la durabilité du code.

4.1 Les couches de l'application

- **UI Layer** : UI Layer (JavaFX/FXML) : contient les contrôleurs (ArtistController, ArtworkController, GalleryController...). Cette couche se limite à gérer les interactions utilisateur et délègue toute logique métier à la couche service.
- **Service Layer** : Service Layer (Logique Métier) : classes telles que DatabaseArtistService, DatabaseGalleryService, DatabaseWorkshopService. Point d'entrée unique selon le pattern Singleton via le ServiceProvider. Cette couche orchestre les règles métier et coordonne les appels DAO.
- **DAO Layer** : DAO Layer (Abstraction JDBC) : implémentations JDBC telles que JdbcArtistDao, JdbcGalleryDao, JdbcWorkshopDao. Cette couche traduit les objets Java en requêtes SQL et inversement.
- **Connexion** : ConnectionManager / DatabaseConfig : gestionnaire centralisé de la connexion JDBC, s'appuyant sur DriverManager.getConnection() avec les paramètres définis dans DatabaseConfig.
- **MySQL Backend** : MySQL Backend : la base ArtConnect avec ses 14 tables, ses triggers, ses vues, ses index et ses procédures stockées.

4.2 Pattern Singleton dans le ServiceProvider

La classe ServiceProvider instancie chaque service une seule fois en tant qu'attribut static final. Ce pattern Singleton garantit qu'une seule connexion logique par service est gérée pendant toute la durée de vie de l'application, réduisant ainsi la création inutile d'objets et la consommation mémoire.

```
private static final ArtistService artistService = new
DatabaseArtistService();
```

5. Sécurité : principes et décisions techniques

La sécurité est une préoccupation transversale intégrée à chaque niveau de l'architecture, depuis la couche base de données jusqu'à la couche applicative Java.

5.1 Protection contre les injections SQL : PreparedStatement

La menace la plus critique pour une application connectée à une base de données est l'injection SQL. Elle consiste à insérer du code SQL malveillant dans un paramètre d'entrée afin de manipuler les requêtes exécutées côté serveur.

Méthode vulnérable (à ne pas faire)

La concaténation directe de paramètres utilisateur dans une requête SQL expose l'application à des attaques :

```
String sql = "SELECT * FROM Artist WHERE city = '" + city + "'";
```

Un attaquant pourrait soumettre la valeur : ' OR '1'='1 et obtenir l'intégralité de la table Artists.

Méthode sécurisée implémentée dans le projet

Le projet utilise systématiquement des PreparedStatement, qui séparent structurellement la requête SQL de ses paramètres. Les paramètres sont transmis comme des données et non comme du code SQL :

```
String sql = "SELECT * FROM Artist WHERE city = ?";
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, city);
```

Cette pratique est appliquée dans toutes les méthodes d'écriture des DAOs (save, update, delete) ainsi que dans les requêtes paramétrées de lecture. On la retrouve notamment dans JdbcArtistDao, JdbcWorkshopDao, JdbcArtworkDao et l'ensemble des implémentations de persistance.

5.2 Gestion sécurisée des ressources : try-with-resources

Les connexions JDBC, les PreparedStatement et les ResultSet sont des ressources système critiques. Une fermeture incorrecte peut entraîner des fuites mémoire, des connexions zombies ou une saturation du serveur MySQL.

Le projet utilise systématiquement le bloc try-with-resources introduit en Java 7, qui garantit la fermeture automatique des ressources même en cas d'exception :

```
try (Connection conn = getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    // utilisation des ressources
} // fermeture automatique garantie
```

Ce pattern est appliqué dans l'ensemble des méthodes DAO du projet, pour les objets Connection, PreparedStatement et ResultSet.

5.3 Intégrité des données côté base de données : les triggers

La sécurité fonctionnelle ne repose pas uniquement sur l'application Java — elle est également renforcée directement dans MySQL via des triggers, qui constituent un filet de sécurité indépendant de la couche applicative.

Trigger `trg_check_workshop_capacity`

Ce trigger s'exécute avant chaque insertion dans la table `Booking`. Il compte les participants déjà inscrits à un atelier et compare ce nombre à la capacité maximale. Si l'atelier est complet, l'insertion est rejetée avec une erreur SQL explicite :

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Workshop full';
```

Cette vérification côté base de données est critique : même si un bug applicatif contournait la vérification Java, la règle serait quand même appliquée.

Trigger `trg_check_exhibition_dates`

Ce trigger garantit la cohérence temporelle des expositions en rejetant toute insertion où la date de fin serait antérieure ou égale à la date de début. Il évite des incohérences métier qui pourraient conduire à des erreurs d'affichage ou de calcul.

5.4 Séparation des responsabilités et principe de moindre privilège

L'architecture en couches garantit que chaque composant n'accède qu'aux données dont il a besoin. Les contrôleurs JavaFX n'ont aucun accès direct à la base de données : ils passent systématiquement par les services, qui délèguent aux DAOs. Cette séparation réduit la surface d'attaque et facilite l'audit de sécurité.

5.5 Point d'amélioration identifié : gestion des credentials

Le fichier `DatabaseConfig.java` contient actuellement les identifiants de connexion MySQL en dur dans le code source (URL, USER, PASSWORD). Cette pratique est acceptable dans un contexte académique de développement local, mais constituerait une vulnérabilité sérieuse en environnement de production. Une amélioration recommandée serait de stocker ces paramètres dans un fichier de configuration externe non versionné (type `.env` ou `application.properties` exclu du dépôt Git via `.gitignore`).

6. Numérique durable (Green IT)

L'éco-responsabilité numérique, ou Green IT, consiste à réduire l'empreinte environnementale des systèmes informatiques en optimisant leur consommation de ressources : CPU, mémoire, réseau, énergie. Le projet ArtConnect Pro intègre cette préoccupation à plusieurs niveaux.

6.1 Optimisation des requêtes SQL par les index

Sans index, MySQL effectue un "full table scan" pour chaque recherche, ce qui signifie qu'il parcourt l'intégralité de la table même pour une requête ne retournant qu'un seul résultat. Sur de grands volumes de données, cela se traduit par une consommation CPU et I/O excessive.

Le projet définit quatre index stratégiques, choisis en fonction des cas d'usage réels de l'application :

- **idx_artist_city** : idx_artist_city : accélère les recherches géographiques d'artistes, fonctionnalité centrale de l'application.
- **idx_artwork_status** : idx_artwork_status : optimise le filtrage des œuvres par statut (FOR_SALE, SOLD, EXHIBITED).
- **idx_workshop_date** : idx_workshop_date : accélère les requêtes de calendrier sur les ateliers, ordonnées par date.
- **idx_booking_workshop** : idx_booking_workshop : optimise les jointures entre la table Booking et Workshop, essentielles pour le calcul des taux de remplissage.

Ces index réduisent directement la charge CPU du serveur MySQL et diminuent le temps de réponse des requêtes, ce qui se traduit par une consommation énergétique moindre du serveur.

6.2 Réduction du trafic réseau par les vues et procédures stockées

Les vues SQL et les procédures stockées permettent de déporter une partie du traitement directement sur le serveur MySQL, réduisant ainsi le volume de données transmises sur le réseau entre l'application Java et la base de données.

Vues créées

- **view_artwork_artist** : view_artwork_artist : agrège en une seule requête les informations d'une œuvre et de son artiste, évitant deux requêtes séparées côté Java.
- **view_workshop_bookings** : view_workshop_bookings : calcule à la volée le nombre de réservations par atelier, via un COUNT côté serveur.
- **view_public_exhibitions** : view_public_exhibitions : joint les expositions avec les galeries en une requête précalculée.

Procédures stockées

- **book_workshop** : book_workshop(p_member, p_workshop) : encapsule la logique d'insertion d'une réservation côté serveur, limitant les allers-retours réseau.
- **add_artwork** : add_artwork(...) : gère atomiquement l'insertion d'une œuvre et son association à un artiste, en une seule transaction côté MySQL.

- **get_workshop_participants** : `get_workshop_participants(p_workshop)` : fonction scalaire retournant directement un entier, évitant de transférer un ResultSet complet vers Java pour un simple comptage.

6.3 Prévention des fuites mémoire par try-with-resources

Comme mentionné dans la section sécurité, l'utilisation du try-with-resources JDBC n'est pas seulement une bonne pratique de sécurité — c'est aussi un geste en faveur du numérique durable. Une connexion JDBC non fermée continue de consommer des ressources côté serveur MySQL (fil d'exécution, mémoire allouée, verrous potentiels). En garantissant la fermeture systématique de chaque ressource, le projet évite la prolifération de connexions zombies qui gaspilleraient des ressources serveur.

6.4 Architecture découplée et dette technique réduite

La séparation stricte en couches (UI / Service / DAO / BDD) réduit la dette technique du projet. Un code bien structuré est un code plus facile à maintenir, à faire évoluer et à optimiser. Il nécessite moins de refactoring coûteux, réduit les risques de régression et permet aux développeurs d'intervenir efficacement sans avoir à tout réécrire.

Du point de vue Green IT, un code de qualité dure plus longtemps. Il évite les cycles de réécriture complète qui impliquent de nouveaux serveurs, de nouvelles instances de base de données et une reconsommation de ressources.

6.5 Migration de InMemory vers MySQL : un gain durable

Le point de départ du projet — remplacer un système InMemory par MySQL — est lui-même un choix durable. Un stockage en mémoire ne survit pas aux redémarrages de l'application, ce qui force des rechargements de données permanents et rend impossible toute optimisation à long terme. Une base de données persistante permet la mise en place de caches, d'index, de vues matérialisées et d'optimisations progressives, contribuant à réduire la charge de traitement sur la durée.

7. Fonctionnalités SQL avancées

Au-delà du schéma relationnel de base, le projet implémente un ensemble de fonctionnalités SQL avancées qui illustrent la maîtrise du moteur MySQL et renforcent les objectifs de sécurité et de performance.

7.1 Récapitulatif des fonctionnalités

Type	Nom	Rôle
Vue	view_artwork_artist	Synthèse œuvres et artistes
Vue	view_workshop_bookings	État des réservations par atelier
Vue	view_public_exhibitions	Agenda public des galeries
Trigger	trg_check_workshop_capacity	Rejet si atelier complet
Trigger	trg_check_exhibition_dates	Validation cohérence des dates
Index	idx_artist_city	Optimisation recherche géographique
Index	idx_artwork_status	Filtrage par disponibilité
Index	idx_workshop_date	Optimisation calendrier
Index	idx_booking_workshop	Jointures rapides réservations
Procédure	book_workshop()	Logique de réservation atomique
Procédure	add_artwork()	Insertion sécurisée d'œuvre + liaison
Fonction	get_workshop_participants()	Comptage participants (retourne INT)

7.2 Mapping JDBC : du relationnel vers l'objet

Le passage des données de MySQL vers les objets Java est réalisé dans chaque DAO via la méthode `mapXxx(ResultSet rs)`. Cette méthode lit les colonnes du `ResultSet` et les affecte aux attributs de l'objet Java correspondant. Les relations N:N (par exemple Artist → disciplines via Exerce) sont résolues par des sous-requêtes préparées utilisant la même connexion, toujours via `PreparedStatement`.

8. Conclusion et bilan

Le projet ArtConnect Pro illustre comment une application de gestion d'une communauté artistique peut être conçue en intégrant dès l'architecture les deux enjeux majeurs que sont la sécurité et le numérique durable, sans que cela ne se fasse au détriment de la fonctionnalité ou de la maintenabilité.

Réalisations techniques

- Base de données relationnelle complète en 3FN (14 tables, clés étrangères, types ENUM).
- Fonctionnalités SQL avancées : 3 vues, 4 index, 2 triggers, 2 procédures stockées, 1 fonction scalaire.
- Couche JDBC complète connectée à MySQL, avec mapping objet-relationnel manuel.
- Application JavaFX fonctionnelle avec opérations CRUD sur l'ensemble du domaine métier.

Points forts en matière de sécurité

- Utilisation systématique de PreparedStatement : protection totale contre les injections SQL.
- Gestion des ressources par try-with-resources : prévention des fuites mémoire et de connexion.
- Triggers MySQL : intégrité des données garantie indépendamment de la couche applicative.
- Architecture découplée : isolation des responsabilités, réduction de la surface d'attaque.

Points forts en matière de Green IT

- Index SQL ciblés : réduction de la charge CPU serveur pour les requêtes fréquentes.
- Vues et procédures stockées : déport du traitement côté serveur, réduction du trafic réseau.
- try-with-resources JDBC : libération immédiate des ressources serveur après chaque opération.
- Architecture propre et découplée : code maintenable sur le long terme, réduisant la dette technique.

Perspectives d'amélioration

- Externaliser les credentials de connexion dans un fichier de configuration non versionné.
- Implémenter un pool de connexions (HikariCP) pour optimiser davantage les ressources JDBC.
- Ajouter des logs d'audit pour la traçabilité des opérations sensibles.
- Envisager le chiffrement des données sensibles (emails, téléphones) au repos.

« *Un code propre consomme moins de ressources et dure plus longtemps* »